SAM3D: Zero-Shot 3D Object Detection via Segment Anything Model

Dingyuan Zhang[1], Dingkang Liang[1], Hongcheng Yang[1],
Zhikang Zou[2], Xiaoqing Ye[2], Zhe Liu[1], Xiang Bai[1†]
[1]Huazhong University of Science and Technology, {dyzhang233, dkliang, xbai}@hust.edu.cn
[2]Baidu Inc., China

https://github.com/DYZhang09/SAM3D

arXiv:2306.02245v2 [cs.CV] 29 Jan 2024

1. Introduction

In the past few years, foundation models have thrived and succeeded in linguistic and visual tasks, showing astonishing zero-shot and few-shot capabilities. Their advances encourage researchers and industries to extend the boundaries of what artificial intelligence can do and have shown some fantastic products (e.g., ChatGPT [1]) with the potential to change the world.

Recently, Kirillov et al. [11] proposed a new vision foundation model for image segmentation, the Segment Anything Model (SAM), trained on a huge dataset called SA-1B. The flexible prompting support, ambiguity awareness, and vast training data endow the SAM with powerful generalization, enabling the ability to solve downstream segmentation problems using prompt engineering. Some following works leverage the excellent zero-shot capability of SAM to solve other 2D vision tasks (e.g., medical image processing [25] and camouflaged object segmentation [20]). Although SAM presents great power on some 2D vision tasks, whether it can be adapted to 3D vision tasks still needs to be discovered. With this inspiration, a few works attempt to combine SAM with pre-trained 3D models to learn 3D scene representation (e.g., SA3D [2]) and single-view reconstruction (e.g., anything-3D [15]), showing promising results.

3D object detection, one of the fundamental tasks in 3D vision, has a wide range of real-world applications (e.g., autonomous driving). Although plenty of works aim to solve this task, the zero-shot setting on 3D object detection still needs to be explored. Thus, considering the advance of SAM, it is natural to question: *Can we adapt the zero-shot capability of SAM to 3D object detection*?

In this paper, we aim to explore the zero-shot 3D object detection with SAM [11] alone. Considering SAM is initially built for 2D images, many challenges exist when using SAM for 3D detection (Please refer to the appendix for more discussion). The key insight is that we can leverage the powerful capability of SAM for 3D object detection by using the Bird's Eye View (BEV), which carries crucial 3D information (e.g., depths) with a 2D image-like data format. Thus, the challenges to using SAM for 3D detection can be significantly solved. With this observation, we present SAM3D, which uses SAM to segment on BEV maps and predicts objects based on the masks from its outputs.

We evaluate our method on the large-scale Waymo Open Dataset [19], and the results show the great potential of SAM on 3D object detection. Although this paper is only an early attempt, it gives a positive signal for applying vision foundation models like SAM for 3D vision tasks, especially for 3D object detection.

2. Method

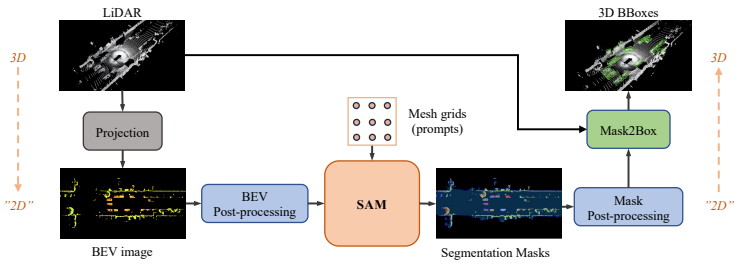
We consider point cloud as the input of our method, which is a 3D representation and naturally sparse, while SAM is trained for 2D images with dense semantics. Our basic idea is to translate LiDAR points into a 2D image-like representation with 3D information that narrows the domain gap, thus BEV is a straightforward choice. We build the whole pipeline with SAM based on BEV, shown as Fig. 1(a). Our method mainly contains five steps:

Firstly, our method conducts the LiDAR-to-BEV projection, which translates sparse LiDAR signals to discriminative BEV images. At this step, we use the projection equations to determine each point's coordinate on the image plane and a predefined intensity-to-RGB mapping to get RGB vectors for pixels in a BEV image, making it more discriminative during processing.

Then, the BEV post-processing modifies the original BEV images with the morphology dilation (interpreted as a max pooling) since SAM is trained on natural images with "dense" signals, which differs from the "sparse" BEV images. This step helps form more suitable inputs for SAM, leading to easier segmentation and better performance.

After obtaining the desired BEV images, we segment the BEV images using SAM, which supports various prompts

†Corresponding author.



(a)

Version	Level 1		Level 2	
	AP	APH	AP	APH
ViT-B	17.30	11.58	16.90	11.31
ViT-L	**19.94**	13.25	**19.48**	12.93
ViT-H	19.51	**13.30**	19.05	**12.98**

(b)

Pillar size	Level 1		Level 2	
	AP	APH	AP	APH
0.05m	16.45	11.20	16.30	11.11
0.075m	18.70	12.59	18.25	12.29
0.1m	**19.51**	**13.30**	**19.05**	**12.98**
0.125m	19.25	13.12	18.80	12.81
0.15m	18.16	12.42	17.73	12.11
0.175m	16.16	10.88	15.77	10.60
0.2m	14.06	9.41	13.73	9.18
0.4m	~0	~0	~0	~0

(c)

Figure 1: (a) The overall framework of our method. We first project LiDAR points to colorful BEV images via a predefined palette, then post-process BEV images to better fit the requirements of SAM. After the segmentation, we post-process the noisy masks and finally predict 3D bounding boxes with the aid of LiDAR points. (b) The results of SAM3D using different versions of SAM. (c) The results of SAM3D using different pillar sizes. We report metrics of VEHICLE in the range [0,30) on Waymo *validation* set.

like point, box, and mask prompts. Our goal in this step is to segment foreground objects as many as possible, so we choose to cover the whole image with mesh grid prompts. Additionally, we prune the prompts in this step without performance sacrifice to accelerate the segmentation.

Despite SAM's powerful zero-shot capability, a non-negligible domain gap still exists. Hence, we propose mask post-processing for filtering noisy masks according to some rules drawn from priors, which reduces the number of false positives and helps improve the final performance.

Finally, after the segmentation and post-processing, we predict 3D bounding boxes from the foreground masks. Since BEV images already carry depth information, we can directly estimate the horizontal attributes (i.e., horizontal object center, length, width, and heading) of 3D bounding boxes from the 2D masks. Meanwhile, for the vertical attributes (i.e., vertical object center and height), LiDAR points will be utilized as extra information compensation.

Please refer to the appendix for more detailed methods.

3. Experiments

We evaluate our method on the Waymo Open Dataset [19], one of the large-scale datasets for autonomous driving. The dataset is split into 798 training sequences, 202 validation sequences, and 150 testing sequences. Since our method performs zero-shot object detection, we only focus on the validation sequences. For the metrics, because of the natural sparsity of point clouds and the lack of semantic label outputs, we only care about the mAP and mAPH of VEHICLE with a distance of at most 30 meters in this paper.

Since SAM uses different backbones with different complexities, we conduct experiments to evaluate the effectiveness of our method, shown in Fig. 1(b). It reveals that using SAM with less capacity performs worse. However, there is only a marginal difference between SAM with ViT-L and ViT-H. We argue that the model capacity is not the performance bottleneck when using large models, and the power of SAM still needs to be fully unleashed. For insurance purposes, we use SAM with ViT-H. We also conduct experiments to determine how the pillar size influences the performance in Fig. 1(c). When using larger pillar sizes such as $0.2m$ and $0.4m$, the discretization errors are relatively large, and it is hard to distinguish different objects when they are close to each other. However, pillar sizes that are too small also harm performance. One possible reason is that due to the high resolution of the small pillar size and the sparsity of LiDAR signals, it is difficult for individual instances to form a completely connected region. SAM tends to separate one object into many parts. We set the pillar size as $0.1m$, which is a good balance. Please refer to the appendix for all detailed results.

4. Conclusion

This paper explores the zero-shot 3D object detection with the visual foundation model SAM and proposes the SAM3D. To narrow the gap between the training data of SAM and 3D LiDAR signals, we use the BEV images to represent 3D outdoor scenes. We propose a SAM-powered BEV processing pipeline to utilize the great zero-shot capability of SAM for zero-shot 3D object detection. Qualitative and ablation experiments on the Waymo open dataset

show promising results for adapting the zero-shot ability of SAM to 3D object detection. Although this paper is only an early attempt, we believe it presents a possibility and opportunity to unleash the power of foundation models like SAM on 3D tasks with technologies like few-shot learning, model distillation, and prompt engineering in the future. The code has been released in `https://github.com/DYZhang09/SAM3D`.

5. Acknowledgements

This work was supported in part by the National Science Fund for Distinguished Young Scholars of China (Grant No. 62225603), in part by the Hubei Key R&D Program (Grant No. 2022BAA078), and part by the "Qisun Ye" Science Fund (U2341227).

6. Appendix

6.1. Related Work

6.1.1 2D tasks with SAM

Kirillov et al. [11] proposed a new vision foundation model for image segmentation, the Segment Anything Model (SAM), trained on a huge dataset called SA-1B to solve the segment anything task. The flexible prompting support, ambiguity awareness, and vast training data endow the SAM with powerful generalization, enabling the ability to solve downstream segmentation problems using prompt engineering, inspiring many following works. SAMPolyp [25] applies SAM to the polyp segmentation task under the unprompted settings. Deng et al. [6] assess the zero-shot segmentation performance of the SAM model on digital pathology tasks and find that SAM achieves remarkable segmentation performance for large connected objects. He et al. [7] test SAM's accuracy in 12 medical image datasets, revealing that SAM is more accurate in 2D medical images, larger target region sizes, and easier cases. SAMCOD [20] evaluates SAM's performance on camouflaged object detection (COD) benchmarks, indicating that SAM can achieve a noteworthy performance compared to some COD-oriented methods. Ji et al. [10] compare SAM with cutting-edge methods and observe the limited power of SAM in concealed scenes. These works focus on utilizing the SAM on 2D tasks like medical image analysis and camouflaged object detection, while our method explores the ability of SAM on the 3D perception task.

6.1.2 3D tasks with SAM

Although SAM presents great power on some 2D vision tasks, whether it can be adapted to 3D vision tasks still needs to be discovered. With this inspiration, a few works attempt to combine SAM with pre-trained 3D models. SA3D [2] generalizes SAM to segment 3D objects by leveraging the Neural Radiance Field (NeRF) as a cheap and off-the-shelf prior, which constructs the 3D mask of the target object via alternately performing the proposed mask inverse rendering and cross-view self-prompting across various views. Anything-3D [15] combines BLIP, a pre-trained 2D text-to-image diffusion model, and SAM for the single-view conditioned 3D reconstruction task, showing promising results. 3D-Box-Segment-Anything [3] utilizes SAM with a pre-trained 3D detector VoxelNeXt [4] for interactive 3D detection and labeling. Unlike them, our method explores zero-shot 3D object detection with SAM alone (i.e., it DOES NOT rely on any other pre-trained model).

6.1.3 3D object detection

3D object detection, one of the fundamental tasks in 3D vision, has a wide range of real-world applications (e.g., autonomous driving). There are plenty of works [22, 12, 16, 4, 13, 24, 26, 21] aim to solve this task. SECOND [22] and PointPillars [12] convert the point cloud into grid-based representation and introduce the sparse convolution and pillars representation, respectively, to solve the 3D object detection with low time consumption. PointRCNN [18] proposes to process the point cloud directly and segment the point cloud before the second-stage refinement for generating high-quality proposals. PVRCNN [16] combines 3D CNN and point-based operations to learn more representative features. VoxelNeXt [4] introduces a fully sparse voxel-based pipeline for 3D object detection without relying on hand-crafted proxies, achieving a better speed-accuracy trade-off. QTNet [9] proposes a new query-based temporal fusion method to facilitate temporal detection efficiency. ViT-WSS3D [24] offers a novel weakly semi-supervised paradigm to lower the dependencies on expensive 3D annotations. Although 3D object detection with 3D annotations has been widely studied, the zero-shot setting on 3D object detection, which has significant practical value (e.g., low-cost data labeling), still needs to be explored. In this paper, we aim to explore zero-shot 3D object detection with the aid of SAM.

6.2. Preliminary

6.2.1 Definitions of 3D object detection

Similar to 2D object detection, the goal of 3D object detection is to predict the locations and categories of all objects of interest given the perceptive sensor data (e.g., LiDAR points for LiDAR-based 3D object detection and camera images for camera-based 3D object detection). In this paper, we focus on the LiDAR-based 3D object detection. To be more specific, given LiDAR points $P = \{(x_i, y_i, z_i)\}_{i=1}^{N_p}$ of the scene, 3D detectors need to infer about all objects $O = \{(c_i, B_i^{3D})\}_{i=1}^{N_o}$ in the scene, where c_i and B_i^{3D} are

the category and geometric attributes of the i-th object, and N_o is the number of objects. Typically, we define the geometric attributes as object centers, dimensions, and orientations, formally written as:

$$B_i^{3D} = (x_i^{3D}, y_i^{3D}, z_i^{3D}, dx_i^{3D}, dy_i^{3D}, dz_i^{3D}, \theta_i^{3D}), \quad (1)$$

where $(x_i^{3D}, y_i^{3D}, z_i^{3D})$, $(dx_i^{3D}, dy_i^{3D}, dz_i^{3D})$, and θ_i^{3D} are the 3D center, dimension, and orientation of the i-th object, respectively.

6.2.2 Challenges of 3D object detection using SAM

Since SAM was initially trained for 2D segmentation with natural images, many inherent challenges exist to adopting SAM for LiDAR-based 3D object detection. In this section, we provide a more in-depth discussion of these challenges, which can be categorized into the following parts:

The input data of SAM and 3D detectors are dramatically different. On the one hand, the formats of input data are different. The original SAM takes 2D images as inputs, which consist of "dense" pixels distributed evenly on the entire 2D image plane. However, for LiDAR-based 3D object detection, the inputs are LiDAR points, representing the location of "sparse" points spread unevenly across the 3D space. On the other hand, the information contained in inputs is different. The original SAM is trained with natural images, with pixels carrying rich semantic information, while LiDAR points only carry the geometrical information of 3D scenes. To narrow the gap of input data, we use the Bird's Eye View (BEV) as the media because of its 2D format and 3D information awareness.

The output data of SAM and 3D detectors are significantly distinct. SAM outputs 2D segmentation masks indicating the possible foreground pixels, while typical 3D detectors output 3D bounding boxes. Translating the 2D segmentation masks into 3D bounding boxes is a pivotal problem. Thanks to the property of outdoor scenes that no objects stack vertically at the same position, we can leverage the BEV maps and input LiDAR points to finish the translation, and finally equipped with SAM to form a 3D detector.

The capability of 3D perception of SAM is limited. In this paper, we aim to explore zero-shot 3D object detection, which means there are no 3D samples for models to train, and since SAM is only trained with 2D images, its 3D perception capability is limited. To overcome this challenge as much as possible, we use the BEV to "disguise" 3D information into 2D form. Moreover, because of the limited 3D capacity of SAM, its outputs will be noisy, and we propose rule-based post-processing to filter noisy masks, which helps improve performance a lot.

6.3. Proposed Method

6.3.1 Overall framework

Our method mainly contains five steps:

- LiDAR-to-BEV projection translates LiDAR signals to BEV images.
- BEV post-processing modifies original BEV images with a simple operation.
- SAM [11] takes in modified BEV images and mesh grid prompts to segment foreground objects in BEV. In order to accelerate the segmentation process, we prune the prompts in this step without performance sacrifice.
- Mask post-processing filters noisy masks according to some rules drawn from priors, which reduces the number of false positives.
- Mask2Box finds the minimum bounding boxes of foreground masks to extract 2D boxes in BEV and then interacts with LiDAR points, predicting the final 3D bounding boxes of objects.

In the following sections, we will describe the detailed designs for each step.

6.3.2 LiDAR-to-BEV projection

The duty of LiDAR-to-BEV projection is to translate N_p LiDAR points $P = \{(x_i, y_i, z_i)\}_{i=1}^{N_p}$ with range $L_x \leq x_i \leq U_x, L_y \leq y_i \leq U_y$ to a single BEV image $I \in \mathbb{R}^{H \times W \times 3}$.

Each point will fall into a grid of BEV image, and the position of grid (cx, cy) is calculated as follow:

$$cx_i = \lfloor (U_x - x_i)/s_x \rfloor, \quad (2)$$

$$cy_i = \lfloor (U_y - y_i)/s_y \rfloor, \quad (3)$$

where s_x, s_y are the pillar size of x and y axes, U_x, U_y are the coordinate upper bounds of x and y axes respectively, and $\lfloor \cdot \rfloor$ means the floor function.

After obtaining the positions of grids, we need to get the values filled into the BEV image. In order to make segmentation easier, we want the BEV image to be discriminative.

One finding is that the reflection intensity of points is useful, which means we can utilize the intensity $R = \{r_i\}_{i=1}^{N_p}$ to form the feature vectors of grids in a BEV image. Concretely, we first regularize the intensity to $[0, 1]$, and then take it to select color vectors from a predefined palette, which can be formally written as:

$$\mathbf{c_i} = \text{Palette}(\text{Norm}(r_i)) \in \mathbb{R}^3, \quad (4)$$

$$I[cx_i, cy_i, :] = \mathbf{c_i}, \quad (5)$$

where $\text{Palette}: \mathbb{R} \rightarrow \mathbb{R}^3$ is the predefined palette used for translating an intensity scalar to an RGB vector.

For those grids without projected points, we simply fill in all zero vectors. Now we get a discriminative BEV image $I \in \mathbb{R}^{H \times W \times 3}$.

6.3.3 BEV post-processing

Since SAM is trained on natural images, which contain "dense" signals and differ from the "sparse" BEV images, we need to post-process BEV images to narrow the gap. We use the morphology dilation in this paper, which can be interpreted as a max pooling, shown as Eq. 6.

$$I' = \text{MaxPool2D}(I), \tag{6}$$

where I' is the BEV image after post-processing.

6.3.4 Segmentation with SAM

Now, we segment the BEV image using SAM, which supports various prompts like point, box, and mask prompts. Our goal in this step is to segment foreground objects as many as possible, so we choose to cover the whole image with mesh grid prompts. Specifically, we create 32×32 mesh grids evenly distributed on the image plane and regard them as point prompts to SAM.

Although this can cover the whole image, it could be more efficient due to the natural sparsity of BEV images, with many prompts falling into empty space. Based on this observation, we prune the prompts. In particular, we project these prompts onto the BEV image, check the neighbor area of each prompt, and then discard prompts with no activated pixels around. This operation accelerates the whole pipeline dramatically, bringing $5\times$ speed up (from 0.4 FPS to 2 FPS on a single NVIDIA GeForce RTX 4090).

At the end of this step, we now get N_m segmentation masks $M = \{m_i \in \mathbb{R}^{H \times W}\}_{i=1}^{N_m}$ from SAM.

6.3.5 Mask post-processing

Despite SAM's powerful zero-shot capability, a non-negligible domain gap still exists. Hence, the masks from SAM are noisy and need further processing.

In scenes of autonomous driving, typical cars have certain areas and aspect ratios, which can be used to filter out some false positives in masks M. In detail, we filter the noisy segmentation masks using an area threshold $[T_l^a, T_h^a]$ and an aspect ratio threshold $[T_l^r, T_h^r]$. With these operations, we finally obtain N_o relative high-quality foreground masks $M' = \{m_i \in \mathbb{R}^{H \times W}\}_{i=1}^{N_o}$, each mask corresponds to a foreground object.

6.3.6 Mask2Box

After the segmentation, we need to predict 3D bounding boxes B^{3D} from the foreground masks M'. Since BEV images already carry depth information, we can directly estimate the horizontal attributes (i.e., horizontal object center, length, width, and heading) of 3D bounding boxes from the 2D masks. Meanwhile, for the vertical attributes (i.e., vertical object center and height), LiDAR points will be utilized as extra information compensation.

We first extract the 2D minimum bounding boxes from masks, defined as Eq. 7.

$$B^{2D} = \{(x_i^{2D}, y_i^{2D}, dx_i^{2D}, dy_i^{2D}, \theta_i^{2D})\}_{i=1}^{N_o}, \tag{7}$$

where (x_i^{2D}, y_i^{2D}), (dx_i^{2D}, dy_i^{2D}), and θ_i^{2D} are the 2D center, dimension, and rotate angle of the i-th object. N_o is the number of objects.

Then, we project these 2D attributes back to corresponding 3D attributes:

$$x_i^{3D} = U_x - (x_i^{2D} + 0.5) \times s_x, \tag{8}$$

$$y_i^{3D} = U_y - (y_i^{2D} + 0.5) \times s_y, \tag{9}$$

$$dx_i^{3D} = dx_i^{2D} \times s_x, \tag{10}$$

$$dy_i^{3D} = dy_i^{2D} \times s_y, \tag{11}$$

$$\theta_i^{3D} = \theta_i^{2D}, \tag{12}$$

where U_x, U_y are the point cloud ranges and s_x, s_y are the pillar size, defined in Sec. 6.3.2.

Finally, we estimate the vertical centers and heights with LiDAR points. The main idea is that we select points whose BEV projections are inside the 2D bounding boxes and calculate the vertical attributes using their vertical coordinates:

$$Z_i = \{z_j | (x_j, y_j, z_j) \textbf{ inside } B_i^{3D}\}, \tag{13}$$

$$dz_i^{3D} = \max(Z_i) - \min(Z_i), \tag{14}$$

$$z_i^{3D} = \min(Z_i) + \frac{dz_i^{3D}}{2}. \tag{15}$$

6.4. Experiments

6.4.1 Hyperparameters

We set the point cloud range $L_x = L_y = -30.0m, U_x = U_y = 30.0m$ and the pillar size $s_x = s_y = 0.1m$ by default. We use a 3×3 kernel for the dilation in BEV post-processing. For mask post-processing, we set the area thresholds $T_l^a = 200, T_h^a = 5000$ pixels, and the aspect ratio thresholds $T_l^r = 1.5, T_h^r = 4$, respectively. For SAM architecture, we use the default version (ViT-H) with pre-trained weights from its official repository.

6.4.2 Qualitative results

We first show the qualitative results of our method. Fig. 2 shows that relatively high-quality 2D rotated bounding boxes are generated from SAM outputs, indicating SAM's great zero-capability. It means that SAM can generate good masks without touching BEV images and 3D annotations during training. Our mask post-processing and Mask2Box module can translate foreground masks into high-quality 3D bounding boxes. It shows that for those objects that are perceived completely and have distinguishable appearances in BEV images, SAM3D can identify them easily and produce reasonable predictions.

Despite SAM's incredible power, some obvious failure cases still exist: (1) SAM will generate duplicated masks when objects are close to each other (marked as red bubbles in Fig. 2). We argue that it is hard for SAM to identify whether these points belong to different objects or a single large object since it is not trained to handle this situation. (2) Some background objects look similar to cars in BEV images, and SAM sometimes regards them as foregrounds by mistake (marked as blue bubbles in Fig. 2). (3) Due to truncation, occlusion, and the sparsity of LiDAR signals, some cars are partially activated in BEV images. Thus, SAM ignores these objects, leaving many false negatives (marked as white bubbles in Fig. 2). We argue that (2) and (3) are inherent challenges for LiDAR-based 3D object detection and are even more complex for a model trained initially for 2D segmentation tasks. While our method provides some naive solutions, it still demands many efforts (e.g., more powerful background suppression or foreground completion methods can be involved) to solve these problems more perfectly. Moreover, as [10] depicts, SAM struggles to segment camouflaged objects. We can view those objects in (2) and (3) as camouflaged, thus getting poor results. It means we need more powerful technologies to enhance SAM's ability.

6.4.3 Ablation study

We conduct ablation studies to figure out the contribution of different designs. We report the AP and APH for all experiments only for VEHICLE in the range [0, 30).

The effects of pillar size. According to Eq. 2, the pillar size will affect the resolution of BEV images, thus influencing the segmentation results of SAM. In the letter, we report the effects of pillar size. To make its effects more intuitive, we visualize the BEV under different pillar size settings in Fig. 3. When using larger pillar sizes such as $0.2m$ and $0.4m$, the discretization errors are relatively large, and it is hard to distinguish different objects when they are close to each other. However, pillar sizes that are too small also harm the performance. One possible reason is that due to the high resolution of the small pillar size and the sparsity of LiDAR signals, it is difficult for individual instances to form a completely connected region. SAM tends to separate one object into many parts. We set the pillar size as $0.1m$, which is a good balance.

Table 1: The results of SAM3D using different types of BEV images. We report metrics of VEHICLE in range [0,30) on Waymo *validation* set.

BEV type	Level1		Level2	
	AP	APH	AP	APH
Binary	0.94	0.55	0.92	0.53
Intensity	1.93	1.17	1.88	1.14
Intensity + Palette	**19.51**	**13.30**	**19.05**	**12.98**

Table 2: The ablations of post-processes. BEV post. means BEV post-processing. Area and Aspect ratio correspond to filter masks according to areas and aspect ratios in mask post-processing, respectively. We report metrics of VEHICLE in the range [0,30) on Waymo *validation* set.

BEV post.	Area	Aspect ratio	Level1		Level2	
			AP	APH	AP	APH
-	✓	✓	11.01	7.68	10.93	7.64
✓	-	✓	17.52	11.93	17.11	11.65
✓	✓	-	14.05	9.61	13.72	9.38
✓	✓	✓	**19.51**	**13.30**	**19.05**	**12.98**

The effects of reflection intensity. The way to project points into a BEV determines the visual appearance of BEV images, affecting the segmentation. In 6.3.2, we claim that using the reflection intensity of points and a predefined intensity-to-RGB mapping is helpful, and we evaluate its effects in this subsection. We compare our method with two other types: binary and intensity. For binary type, a pixel in a BEV image will be set to white if any point is projected into it, or it will be black otherwise. For intensity type, we use the normalized intensity as the grayscale. In Tab. 1, we can see that using intensity brings gains against the naive binary type, and mapping the intensity to rgb space further dramatically boosts the performance. Because the intensity and RGB mapping make BEV images more discriminative, SAM segments them more easily and precisely, thus improving the performance of the proposed method.

The effects of BEV post-processing. In 6.3.3, we use the morphology dilation to post-process the BEV image for better segmentation. We conduct experiments to figure out its effect in this subsection, shown in the first row of Tab. 2. Without BEV post-processing, it drops about 8% AP and 5% APH on both Level 1 and Level 2. As we have claimed, BEV post-processing helps narrow the gap between BEV and natural images (training data of SAM), leading to better results.

The effects of mask post-processing. In Sec. 6.3.5,

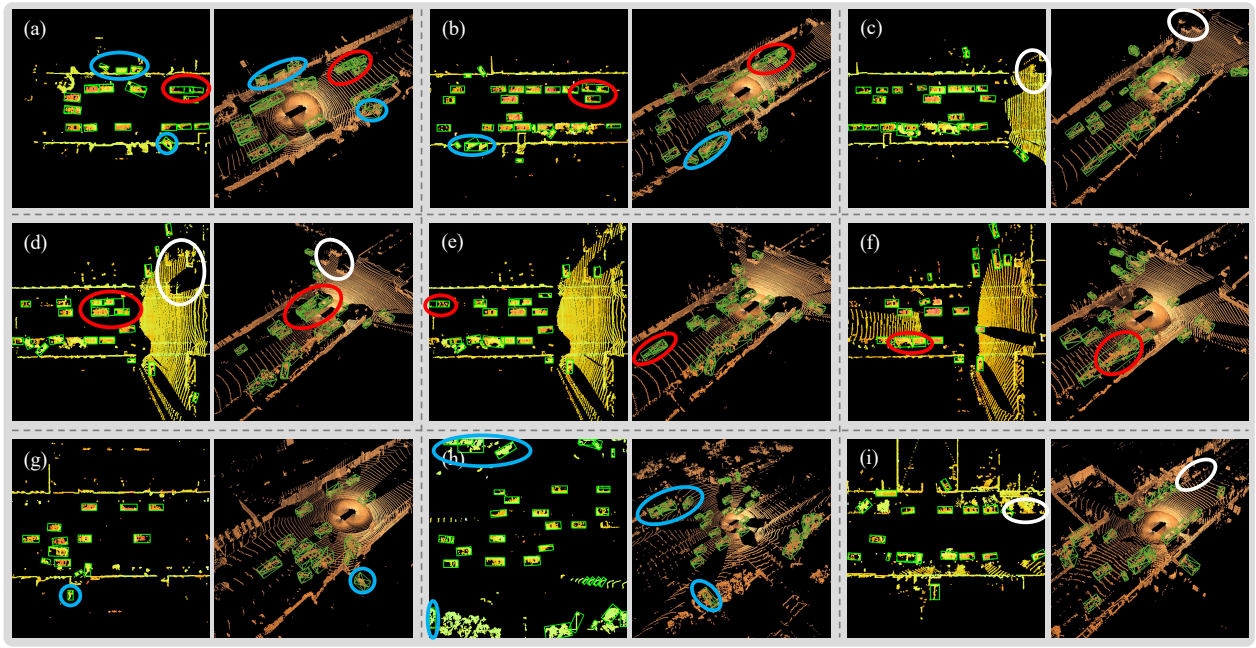


Figure 2: The visualizations of results from SAM3D. Each sub-figure corresponds to a single frame. The left side of each sub-figure is the visualization of 2D bounding boxes under the Bird's Eye View (BEV), and the right is the visualization of 3D bounding boxes.

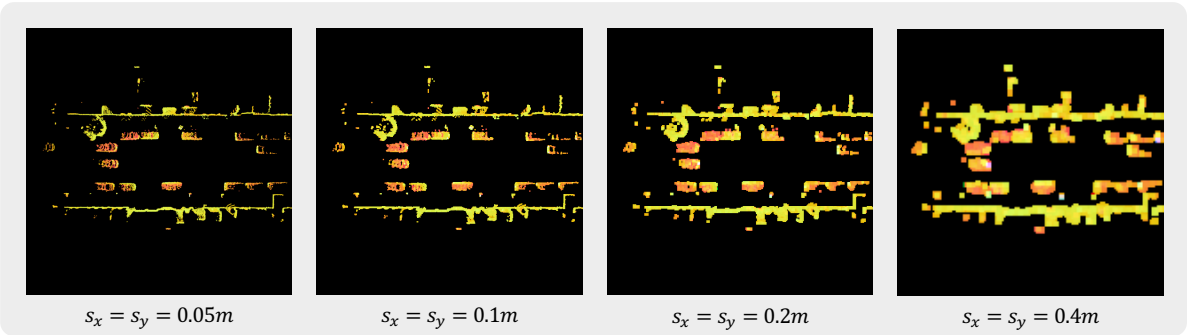


Figure 3: The visualization of BEV images under different pillar size settings.

we propose the mask area and aspect ratio thresholds $(T_l^a, T_h^a), (T_l^r, T_h^r)$ to filter noisy masks from SAM. We also conduct experiments to test its effect, shown in the second and third row of Tab. 2. Since a non-negligible domain gap still exists and the masks from SAM are noisy, it is obvious that all operations in mask post-processing are essential. Dropping any of them will result in a significant performance decrease compared to the full model.

6.4.4 Comparison with fully-supervised 3D object detectors

To better understand the gap between our method and prevailing fully-supervised 3D detectors, we list the results in Tab. 3. Compared with traditional fully-supervised 3D detectors, our method is lagged by a significant gap. It is natural since SAM is only trained for 2D segmentation. Moreover, we observe that the difference between our methods' AP and APH is much more significant than that of others. This is because SAM is not orientation-aware, and the orientation predictions come from the minimum oriented bounding boxes estimation of segmentation masks, which

Table 3: The results of SAM3D and fully-supervised 3D detectors. We report metrics of VEHICLE in range [0,30) on Waymo *validation* set.

Method	Training data	Level1		Level2	
		AP	APH	AP	APH
SECOND [22]	Waymo (3D Det)	85.60	84.94	84.25	83.59
CenterPoint [23]	Waymo (3D Det)	81.93	81.20	80.53	79.82
VoxelRCNN [5]	Waymo (3D Det)	88.85	88.42	87.52	87.10
PVRCNN [16]	Waymo (3D Det)	89.52	88.92	88.19	87.60
PVRCNN++ [17]	Waymo (3D Det)	90.08	89.61	88.77	88.30
CenterFormer [27]	Waymo (3D Det)	91.11	90.58	89.84	89.31
Ours	SA-1B (2D Seg)	19.51	13.30	19.05	12.98

is simple but noisy.

6.5. Discussion

Through the qualitative results and ablation studies, we show that it is possible to leverage SAM, trained on large-scale segmentation datasets without any 3D annotation, to solve the zero-shot object detection task for outdoor scenes.

However, there are some areas for improvement in our current method, and we leave these issues to be solved in the future:

- Utilizing BEV images as representations means our method may be unsuitable for indoor scenes. Finding a better scene representation will be a good solution.
- Due to the occlusion, truncation, and sparsity of LiDAR points, our method generates many false negatives, especially for distant objects. Considering the information from other modalities will be helpful.
- Although we have already reduced the inference time by $5\times$, the inference speed (2 FPS on a single NVIDIA GeForce RTX 4090) is still limited to the complexity of SAM, especially when the number of point prompts is large. Conducting model compression and distillation might solve this problem.
- Our method currently does not support multi-class detection because of the lack of semantic label outputs from SAM. One possible solution is to leverage 3D vision-language models (e.g., CLIP Goes 3D [8], CrowdCLIP [14]) for zero-shot classification.

We believe our method shows the great possibility and opportunity to unleash the potential of foundation models like SAM on 3D vision tasks, especially on 3D object detection. With technologies like few-shot learning and prompt engineering, we can use vision foundation models more effectively to solve 3D tasks better, especially considering the vast difference between scales of 2D and 3D data.

References

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Proc. of Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[2] Jiazhong Cen, Zanwei Zhou, Jiemin Fang, Wei Shen, Lingxi Xie, Xiaopeng Zhang, and Qi Tian. Segment anything in 3d with nerfs. *arXiv preprint arXiv:2304.12308*, 2023.

[3] Yukang Chen, Jianhui Liu, Xiangyu Zhang, Xiaojuan Qi, and Jiaya Jia. 3d-box-segment-anything. `https://github.com/dvlab-research/3D-Box-Segment-Anything.git`, 2023.

[4] Yukang Chen, Jianhui Liu, Xiangyu Zhang, Xiaojuan Qi, and Jiaya Jia. Voxelnext: Fully sparse voxelnet for 3d object detection and tracking. In *Proc. of IEEE Intl. Conf. on Computer Vision and Pattern Recognition*, 2023.

[5] Jiajun Deng, Shaoshuai Shi, Peiwei Li, Wengang Zhou, Yanyong Zhang, and Houqiang Li. Voxel r-cnn: Towards high performance voxel-based 3d object detection. In *Proc. of the AAAI Conf. on Artificial Intelligence*, volume 35, pages 1201–1209, 2021.

[6] Ruining Deng, Can Cui, Quan Liu, Tianyuan Yao, Lucas Walker Remedios, Shunxing Bao, Bennett A Landman, Yucheng Tang, Lee E Wheless, Lori A Coburn, et al. Segment anything model (sam) for digital pathology: Assess zero-shot segmentation on whole slide imaging. In *Medical Imaging with Deep Learning, short paper track*, 2023.

[7] Sheng He, Rina Bao, Jingpeng Li, P Ellen Grant, and Yangming Ou. Accuracy of segment-anything model (sam) in medical image segmentation tasks. *arXiv preprint arXiv:2304.09324*, 2023.

[8] Deepti Hegde, Jeya Maria Jose Valanarasu, and Vishal M Patel. Clip goes 3d: Leveraging prompt tuning for language grounded 3d recognition. *arXiv preprint arXiv:2303.11313*, 2023.

[9] Jinghua Hou, Zhe Liu, Zhikang Zou, Xiaoqing Ye, Xiang Bai, et al. Query-based temporal fusion with explicit motion for 3d object detection. In *Proc. of Advances in Neural Information Processing Systems*, 2023.

[10] Ge-Peng Ji, Deng-Ping Fan, Peng Xu, Ming-Ming Cheng, Bowen Zhou, and Luc Van Gool. Sam struggles in concealed scenes–empirical study on" segment anything". *Sci. China Inf. Sci.*, 2023.

[11] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Porc. of IEEE Intl. Conf. on Computer Vision*, 2023.

[12] Alex H Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. Pointpillars: Fast encoders for object detection from point clouds. In *Proc. of IEEE Intl. Conf. on Computer Vision and Pattern Recognition*, pages 12697–12705, 2019.

[13] Jingyu Li, Zhe Liu, Jinghua Hou, and Dingkang Liang. Dds3d: Dense pseudo-labels with dynamic threshold for

semi-supervised 3d object detection. *Proc. of Intl. Conf. on Robotics and Automation*, 2023.

[14] Dingkang Liang, Jiahao Xie, Zhikang Zou, Xiaoqing Ye, Wei Xu, and Xiang Bai. Crowdclip: Unsupervised crowd counting via vision-language model. In *Proc. of IEEE Intl. Conf. on Computer Vision and Pattern Recognition*, pages 2893–2903, 2023.

[15] Qiuhong Shen, Xingyi Yang, and Xinchao Wang. Anything-3d: Towards single-view anything reconstruction in the wild. *arXiv preprint arXiv:2304.10261*, 2023.

[16] Shaoshuai Shi, Chaoxu Guo, Li Jiang, Zhe Wang, Jianping Shi, Xiaogang Wang, and Hongsheng Li. Pv-rcnn: Point-voxel feature set abstraction for 3d object detection. In *Proc. of IEEE Intl. Conf. on Computer Vision and Pattern Recognition*, pages 10529–10538, 2020.

[17] Shaoshuai Shi, Li Jiang, Jiajun Deng, Zhe Wang, Chaoxu Guo, Jianping Shi, Xiaogang Wang, and Hongsheng Li. Pv-rcnn++: Point-voxel feature set abstraction with local vector representation for 3d object detection. *International Journal of Computer Vision*, 131(2):531–551, 2023.

[18] Shaoshuai Shi, Xiaogang Wang, and Hongsheng Li. Pointrcnn: 3d object proposal generation and detection from point cloud. In *Proc. of IEEE Intl. Conf. on Computer Vision and Pattern Recognition*, pages 770–779, 2019.

[19] Pei Sun, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, et al. Scalability in perception for autonomous driving: Waymo open dataset. In *Proc. of IEEE Intl. Conf. on Computer Vision and Pattern Recognition*, pages 2446–2454, 2020.

[20] Lv Tang, Haoke Xiao, and Bo Li. Can sam segment anything? when sam meets camouflaged object detection. *arXiv preprint arXiv:2304.04709*, 2023.

[21] Kaixin Xiong, Dingyuan Zhang, Dingkang Liang, Zhe Liu, Hongcheng Yang, Wondimu Dikubab, Jianwei Cheng, and Xiang Bai. You only look bottom-up for monocular 3d object detection. *IEEE Robotics and Automation Letters*, 2023.

[22] Yan Yan, Yuxing Mao, and Bo Li. Second: Sparsely embedded convolutional detection. *Sensors*, 18(10):3337, 2018.

[23] Tianwei Yin, Xingyi Zhou, and Philipp Krahenbuhl. Center-based 3d object detection and tracking. In *Proc. of IEEE Intl. Conf. on Computer Vision and Pattern Recognition*, pages 11784–11793, 2021.

[24] Dingyuan Zhang, Dingkang Liang, Zhikang Zou, Jingyu Li, Xiaoqing Ye, Zhe Liu, Xiao Tan, and Xiang Bai. A simple vision transformer for weakly semi-supervised 3d object detection. In *Porc. of IEEE Intl. Conf. on Computer Vision*, pages 8373–8383, 2023.

[25] Tao Zhou, Yizhe Zhang, Yi Zhou, Ye Wu, and Chen Gong. Can sam segment polyps? *arXiv preprint arXiv:2304.07583*, 2023.

[26] Xin Zhou, Jinghua Hou, Tingting Yao, Dingkang Liang, Zhe Liu, Zhikang Zou, Xiaoqing Ye, Jianwei Cheng, and Xiang Bai. Diffusion-based 3d object detection with random boxes. In *Chinese Conference on Pattern Recognition and Computer Vision*, pages 28–40. Springer, 2023.

[27] Zixiang Zhou, Xiangchen Zhao, Yu Wang, Panqu Wang, and Hassan Foroosh. Centerformer: Center-based transformer for 3d object detection. In *Proc. of European Conference on Computer Vision*, 2022.